CORPORACIÓN UNIFICADA NACIONAL DE EDUCACIÓN SUPERIOR - CUN

DISEÑO DE LA ARQUITECTURA DE UN SISTEMA DE GESTIÓN ACADÉMICA (SGA) MEDIANTE GOOGLE AI STUDIO

ACA FINAL - ARQUITECTURA DE SOFTWARE

Estudiante:
Gustavo Grajales

Programa:
Ingeniería de Sistemas

Asignatura:
Arquitectura de Software

Docente:
Alexander Calderón Martinez

Ficha / Grupo:
54451

Fecha:
08 Noviembre de 2026

Enlaces:

Video / pitch en YouTube: https://youtu.be/rMUYrhVCNjE

Repositorio: https://github.com/GustavoGrajales38/ACA-Arquitectura-Software-SGA

1. Introducción y descripción del problema

Una institución de educación superior requiere centralizar en un único Sistema de Gestión Académica (SGA) la información relacionada con estudiantes, docentes, programas, asignaturas, grupos, matrículas, calificaciones y horarios. En la situación actual, parte de estos datos se maneja mediante archivos y aplicaciones independientes, lo que genera duplicidad, demoras en las consultas y dificultades para mantener la trazabilidad de los cambios.

La propuesta presentada en este informe define una arquitectura de software sencilla, segura y mantenible para soportar dicho sistema. No se plantea la implementación completa del SGA; el propósito es modelar una arquitectura coherente con los requerimientos funcionales y no funcionales, explicar sus componentes, representar los procesos principales y establecer mecanismos de validación, trazabilidad y mitigación de riesgos.

2. Objetivos

2.1 Objetivo general

Diseñar y modelar una arquitectura de software para un Sistema de Gestión Académica web que centralice la información institucional y responda de manera coherente a los requerimientos funcionales, no funcionales y atributos de calidad definidos para el caso.

2.2 Objetivos específicos

- Analizar y clasificar los requerimientos funcionales y no funcionales del SGA.
- Definir una arquitectura en capas con organización modular que separe responsabilidades y facilite el mantenimiento.
- Representar la solución mediante modelos de casos de uso, componentes, despliegue, datos y secuencia.
- Relacionar los requerimientos con componentes y evidencias mediante una matriz de trazabilidad.
- Identificar decisiones arquitectónicas, riesgos y mecanismos de validación.
- Utilizar Google AI Studio como herramienta de apoyo, revisando críticamente sus sugerencias.

3. Alcance, actores, supuestos y restricciones

3.1 Alcance

El alcance comprende el diseño arquitectónico de una aplicación web para gestionar autenticación, roles, estudiantes, docentes, programas, asignaturas, grupos, periodos, matrículas, calificaciones, consultas, reportes y auditoría. La propuesta incluye modelos y decisiones de diseño, pero no contempla la implementación completa, pruebas de producción ni despliegue real en infraestructura institucional.

3.2 Actores principales

Actor	Responsabilidad principal
Administrador	Administra usuarios, roles y registros generales del sistema.
Coordinador académico	Gestiona oferta académica, grupos, periodos, matrículas y reportes.
Docente	Consulta sus grupos y registra o actualiza calificaciones autorizadas.
Estudiante	Consulta matrícula, horarios y calificaciones publicadas.

3.3 Supuestos

- Los usuarios cuentan con acceso a un navegador web moderno y conexión a internet.
- La institución dispone de una base de datos relacional o puede contratarla.
- Los roles y permisos institucionales son definidos por personal autorizado.
- Las integraciones con otros sistemas se consideran futuras y se realizarán mediante API REST documentada.

3.4 Restricciones

- El sistema debe ser accesible desde navegadores web.
- Las comunicaciones deben usar HTTPS y las contraseñas no se almacenarán en texto plano.
- La solución debe mantener la separación de responsabilidades y permitir evolución sin rediseño total.
- La actividad se limita al diseño y modelado arquitectónico, no a la implementación completa.

4. Requerimientos funcionales y no funcionales

Los requerimientos suministrados en la guía se organizaron en dos grupos: funcionales (RF), que describen capacidades del sistema, y no funcionales (RNF), que establecen atributos de calidad y restricciones de operación.

ID	Nombre	Síntesis
RF-01	Autenticación	Inicio y cierre de sesión con credenciales individuales.
RF-02	Roles y permisos	Administrador, coordinador, docente y estudiante con funciones restringidas por rol.
RF-03	Gestión de estudiantes	Registrar, consultar, actualizar y desactivar estudiantes.
RF-04	Gestión de docentes	Registrar, consultar, actualizar y desactivar docentes.
RF-05	Programas y asignaturas	Crear y actualizar programas, planes de estudio y asignaturas.
RF-06	Grupos y periodos	Configurar periodos, grupos, cupos, docentes, horarios y aulas/enlaces.
RF-07	Matrícula académica	Matricular estudiantes, validar cupos y evitar duplicados.
RF-08	Registro de calificaciones	Registrar y actualizar calificaciones dentro de los permisos.
RF-09	Consulta de resultados	Consultar asignaturas, horarios y calificaciones publicadas.
RF-10	Reportes	Generar reportes con filtros por periodo y programa.
RF-11	Búsqueda y filtros	Buscar y filtrar registros por identificadores, nombres, programa, periodo o estado.
RF-12	Auditoría	Registrar inicios de sesión, cambios de roles, matrículas y modificaciones de calificaciones.

ID	Atributo	Síntesis
RNF-01	Seguridad	HTTPS, contraseñas protegidas y control por roles.
RNF-02	Rendimiento	95 % de consultas comunes en máximo 3 s con hasta 300 usuarios concurrentes.
RNF-03	Disponibilidad	Objetivo mensual de 99,5 %,

		excluyendo mantenimientos programados.
RNF-04	Escalabilidad	Aumentar recursos y atender crecimiento sin rediseño total.
RNF-05	Mantenibilidad	Separación de responsabilidades, bajo acoplamiento y cohesión adecuada.
RNF-06	Usabilidad y accesibilidad	Interfaz consistente, responsive, comprensible y operable por teclado en funciones principales.
RNF-07	Integridad y consistencia	Validar reglas de matrícula y calificaciones y conservar consistencia ante concurrencia.
RNF-08	Interoperabilidad	Interfaces documentadas, preferiblemente API REST.
RNF-09	Respaldo y recuperación	Copias periódicas y procedimiento de restauración.
RNF-10	Trazabilidad	Relacionar decisiones y requerimientos con componentes y evidencias.
RNF-11	Privacidad	Limitar acceso a datos personales y académicos según rol.
RNF-12	Compatibilidad	Funcionamiento en navegadores recientes y distintos tamaños de pantalla.

5. Arquitectura seleccionada y justificación

Se propone una arquitectura en capas implementada como monolito modular, con un frontend web, una API REST, módulos de negocio claramente separados, una capa de persistencia y una base de datos relacional. Esta alternativa ofrece un equilibrio adecuado entre simplicidad y calidad arquitectónica para el alcance del SGA.

5.1 Capas principales

Presentación: Interfaz web responsable de mostrar información, capturar entradas y adaptarse a diferentes tamaños de pantalla.

API / Aplicación: Expone endpoints REST, recibe solicitudes, aplica autenticación y coordina los servicios del dominio.

Dominio / Servicios: Contiene reglas de negocio de matrículas, calificaciones, grupos, cupos, roles y demás procesos académicos.

Persistencia / Repositorios: Aísla el acceso a datos y concentra consultas y transacciones.

Datos: Base de datos relacional que conserva usuarios, información académica, auditoría y relaciones institucionales.

5.2 Justificación

La arquitectura en capas facilita la mantenibilidad porque evita mezclar interfaz, reglas de negocio y persistencia. El monolito modular reduce la complejidad operativa frente a una arquitectura de microservicios y sigue permitiendo separar funcionalidades por módulos. La API REST favorece la interoperabilidad futura. La base de datos relacional es apropiada porque el dominio académico posee relaciones y reglas de integridad claras entre estudiantes, grupos, asignaturas, matrículas y calificaciones.

6. Modelos del sistema

6.1 Modelo de casos de uso

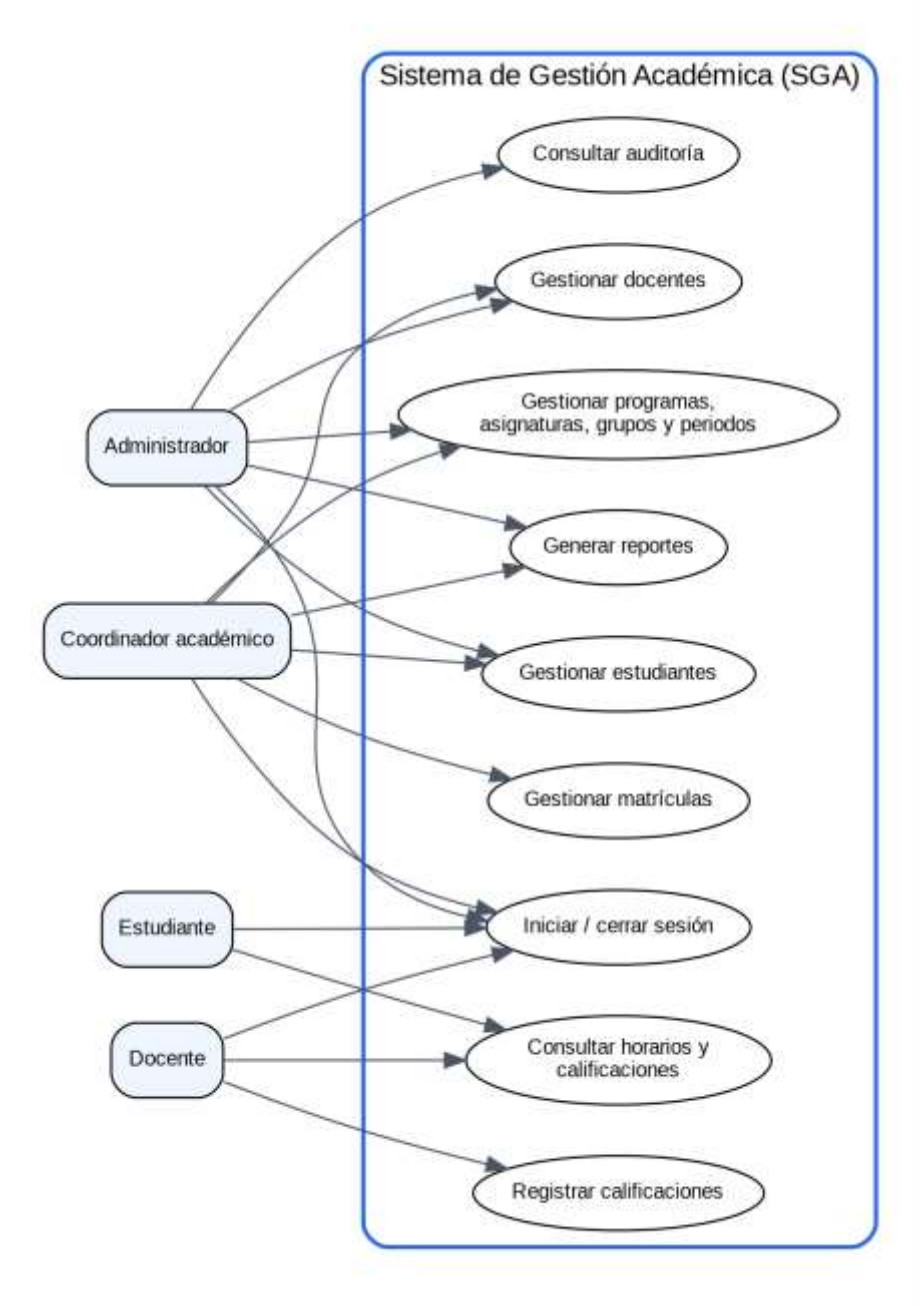


Figura 1. Casos de uso principales del Sistema de Gestión Académica.

El modelo presenta los cuatro actores solicitados y sus interacciones principales. El administrador y el coordinador concentran las funciones de gestión; el docente opera sobre calificaciones y consulta de información; el estudiante utiliza funciones de consulta. Todos requieren autenticación.

6.2 Modelo de componentes / vista lógica

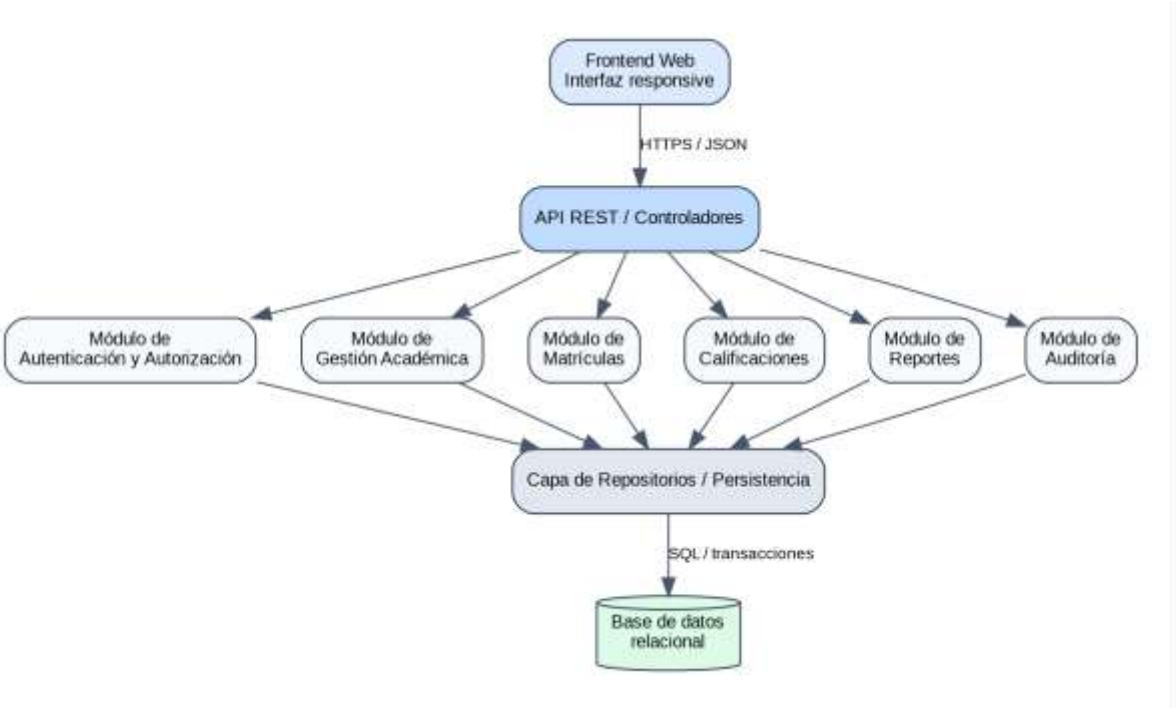


Figura 2. Vista lógica y componentes principales del SGA.

La solución organiza el backend en módulos de autenticación, gestión académica, matrículas, calificaciones, reportes y auditoría. Todos acceden a la persistencia mediante repositorios, lo que reduce el acoplamiento con la base de datos y facilita futuras modificaciones.

6.3 Modelo de despliegue



Figura 3. Despliegue propuesto para la solución web.

El usuario accede desde un navegador mediante HTTPS. El frontend se comunica con la API también mediante un canal seguro. El backend accede a la base de datos mediante una conexión restringida y se contemplan copias periódicas hacia un almacenamiento de respaldo.

6.4 Modelo de datos conceptual

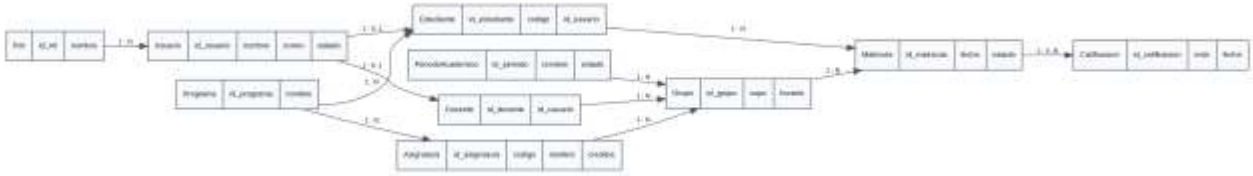


Figura 4. Modelo conceptual de datos y cardinalidades principales.

El modelo conceptual representa las entidades principales indicadas en la actividad. Usuario se relaciona con Rol y puede especializarse como Estudiante o Docente. Los programas contienen asignaturas; las asignaturas se ofrecen en grupos asociados a periodos y docentes; las matrículas vinculan estudiantes con grupos y permiten registrar calificaciones.

6.5 Vista de secuencia: matrícula de estudiante

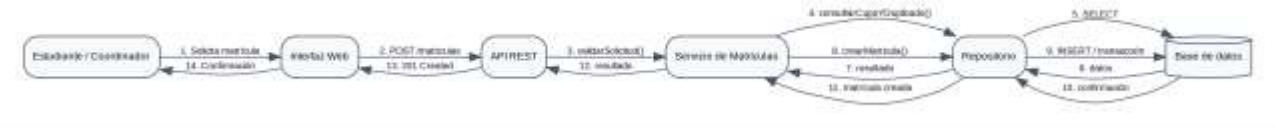


Figura 5. Secuencia simplificada del proceso de matrícula.

El proceso inicia en la interfaz, que envía una solicitud a la API. El servicio de matrículas valida cupo y duplicidad mediante el repositorio. Si las reglas se cumplen, se crea la matrícula dentro de una transacción y la API devuelve una respuesta exitosa. Esta secuencia protege la integridad de los datos al centralizar las reglas en la capa de dominio.

7. Matriz de trazabilidad

La siguiente matriz relaciona cada requerimiento con el componente principal que lo satisface y una evidencia o método de validación. Se mantiene resumida para facilitar su lectura y verificación.

ID	Componente	Modelo asociado	Validación propuesta
RF-01	Autenticación / API	Caso de uso + componentes	Prueba de inicio y cierre de sesión
RF-02	Autenticación y autorización	Casos de uso + componentes	Pruebas de acceso por rol
RF-03	Módulo gestión académica	Casos de uso	CRUD autorizado de estudiantes
RF-04	Módulo gestión académica	Casos de uso	CRUD autorizado de docentes
RF-05	Módulo gestión académica	Componentes + datos	Crear/actualizar programas y asignaturas
RF-06	Módulo gestión académica	Componentes + datos	Configurar grupos, periodos y cupos
RF-07	Módulo matrículas	Secuencia + datos	Validar cupo y duplicidad
RF-08	Módulo calificaciones	Componentes + datos	Prueba de registro con rol docente
RF-09	Frontend + API	Casos de uso	Consulta por estudiante
RF-10	Módulo reportes	Componentes	Generar reportes con filtros
RF-11	API + módulos	Componentes	Pruebas de búsqueda y filtrado
RF-12	Módulo auditoría	Componentes	Verificar registro de eventos
RNF-01	Auth + HTTPS	Despliegue	Revisión TLS y control por roles
RNF-02	API / Backend	Despliegue	Prueba de carga hasta 300 usuarios
RNF-03	Infraestructura	Despliegue	Monitoreo de disponibilidad
RNF-04	Backend / infraestructura	Componentes + despliegue	Escalamiento de recursos
RNF-05	Capas y módulos	Componentes	Revisión de acoplamiento y pruebas
RNF-06	Frontend web	Despliegue	Pruebas responsive y teclado
RNF-07	Dominio + BD	Secuencia + datos	Transacciones y pruebas concurrentes
RNF-08	API REST	Componentes	Documentación de endpoints
RNF-09	BD + backups	Despliegue	Prueba de restauración
RNF-10	Toda la arquitectura	Matriz	Revisión de cobertura RF/RNF
RNF-11	Auth + módulos	Casos de uso	Pruebas de autorización y privacidad
RNF-12	Frontend	Despliegue	Pruebas en navegadores recientes

8. Decisiones arquitectónicas y atributos de calidad

Decisión 1: arquitectura en capas. Se selecciona para separar presentación, negocio y persistencia. Favorece mantenibilidad, pruebas y claridad de responsabilidades.

Decisión 2: monolito modular. Se evita la complejidad de microservicios para un sistema académico de alcance institucional moderado. Los módulos permiten evolución interna sin requerir múltiples despliegues.

Decisión 3: API REST y base de datos relacional. REST facilita interoperabilidad futura y la base relacional conserva integridad y consistencia en entidades altamente relacionadas como matrículas y calificaciones.

Los atributos de calidad priorizados son seguridad, mantenibilidad, rendimiento, disponibilidad, escalabilidad, integridad, interoperabilidad, privacidad, usabilidad y compatibilidad. La propuesta no afirma que estos valores hayan sido medidos en producción; son capacidades de diseño que deberán verificarse mediante pruebas antes de un despliegue real.

9. Análisis de riesgos

Riesgo	Impacto	Mitigación propuesta
Acceso no autorizado	Exposición o modificación de información académica.	HTTPS, contraseñas protegidas, roles, permisos y auditoría.
Sobrecarga en periodos de matrícula	Lentitud o indisponibilidad.	Pruebas de carga, índices, escalamiento de recursos y monitoreo.
Caída o pérdida de la base de datos	Interrupción y pérdida de registros.	Backups periódicos y procedimiento probado de restauración.
Errores o concurrencia en matrículas/calificaciones	Datos duplicados o inconsistentes.	Reglas de negocio, restricciones, transacciones y validación concurrente.
Punto único de falla del backend	Indisponibilidad completa si falla el servidor.	Monitoreo, backups, capacidad de recuperación y futura redundancia si la operación lo exige.

10. Uso crítico de Google AI Studio

Google AI Studio se emplea como herramienta de apoyo para analizar los requerimientos, proponer componentes y detectar riesgos. Sus respuestas se revisan antes de incorporarlas al diseño, de acuerdo con los requerimientos y restricciones definidos para el SGA. Las evidencias de los prompts y resultados utilizados se incorporan mediante capturas legibles.

10.1 Prompts empleados para el análisis arquitectónico

Prompt 1: Analiza los requerimientos funcionales y no funcionales de un Sistema de Gestión Académica para una institución de educación superior. El sistema debe gestionar autenticación, roles, estudiantes, docentes, programas, asignaturas, grupos, matrículas, calificaciones, reportes y auditoría. Propón una arquitectura de software sencilla, segura, mantenible y escalable. Indica qué estilo arquitectónico consideras más apropiado y justifica la elección.

Prompt 2: Ajusta la propuesta anterior. Debes conservar exactamente los 12 requerimientos funcionales y los 12 requerimientos no funcionales definidos en la actividad del Sistema de Gestión Académica, sin agregar requisitos nuevos. Propón una arquitectura en capas con monolito modular, API REST y base de datos relacional. Organiza los componentes principales en: autenticación y autorización, estudiantes, docentes, gestión académica, matrículas, calificaciones, reportes y auditoría. Explica brevemente las responsabilidades y dependencias de cada componente.

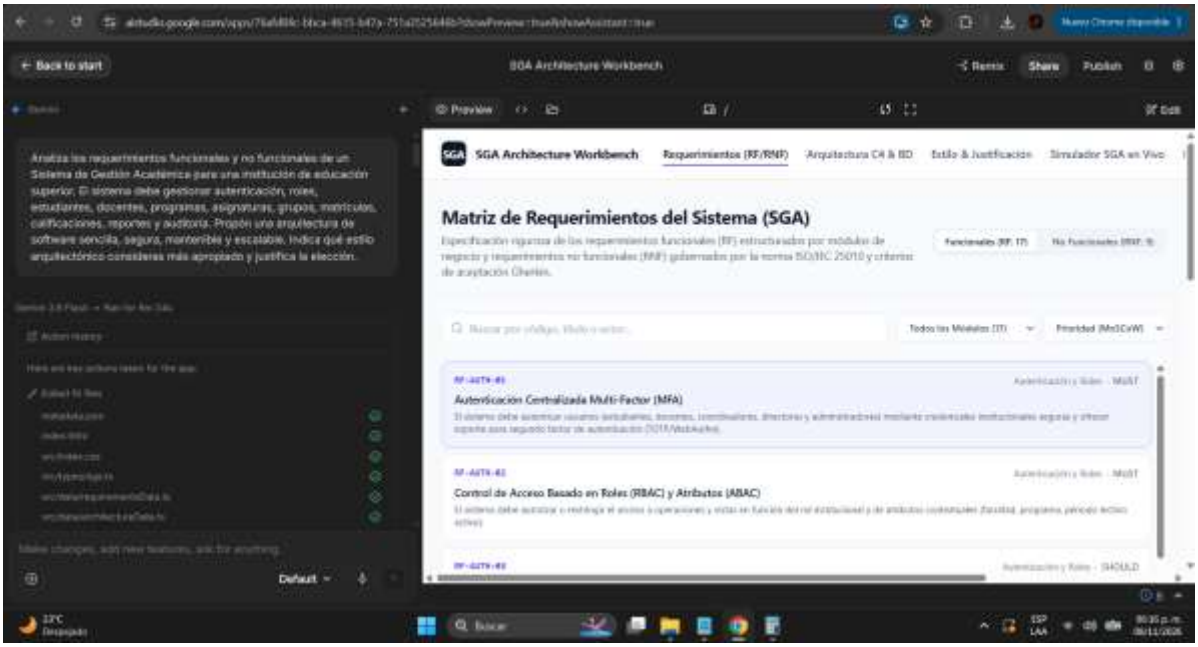
Prompt 3: Con base en la arquitectura propuesta para el Sistema de Gestión Académica, identifica los principales riesgos arquitectónicos, puntos únicos de falla, posibles problemas de seguridad y límites de escalabilidad. Propón estrategias de mitigación para cada riesgo. Además, menciona al menos tres decisiones arquitectónicas justificadas y explica brevemente cómo podrían validarse.

10.2 Evaluación crítica de las sugerencias

Las respuestas de Google AI Studio se revisaron de forma progresiva. La propuesta inicial fue tomada como punto de partida, luego se corrigió para ajustarla exactamente a los requerimientos oficiales del

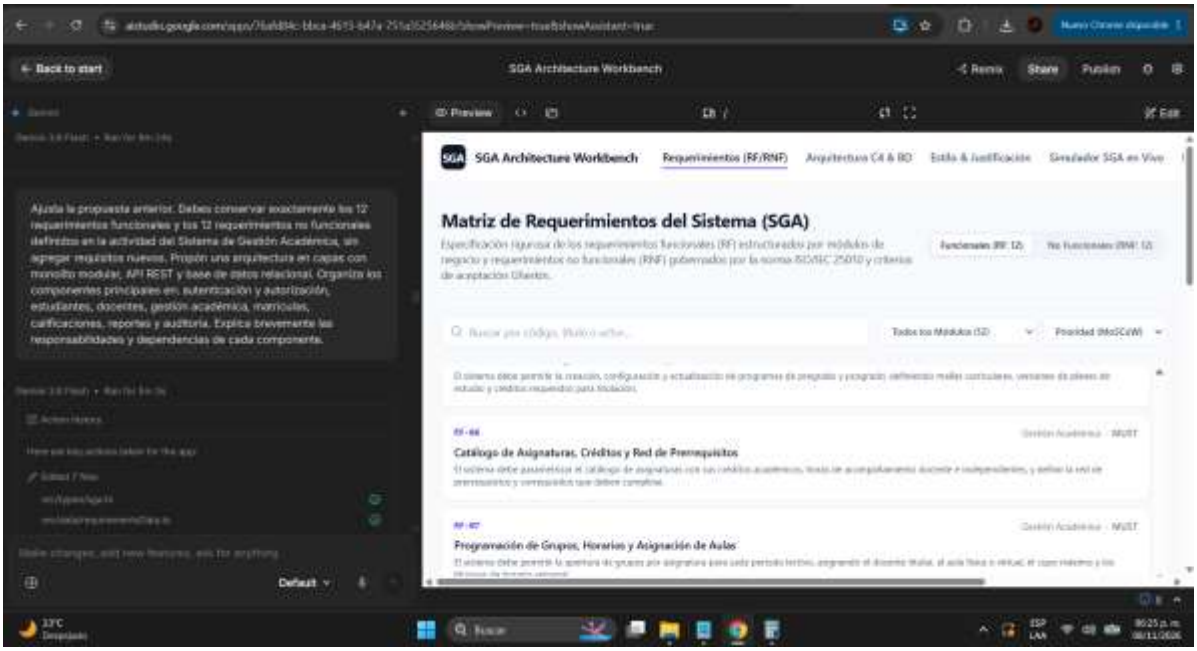
caso y, finalmente, se utilizó la herramienta para identificar riesgos y validar decisiones arquitectónicas. Las siguientes evidencias muestran ese proceso de revisión y ajuste.

Evidencia 1. Propuesta arquitectónica inicial generada en Google AI Studio.



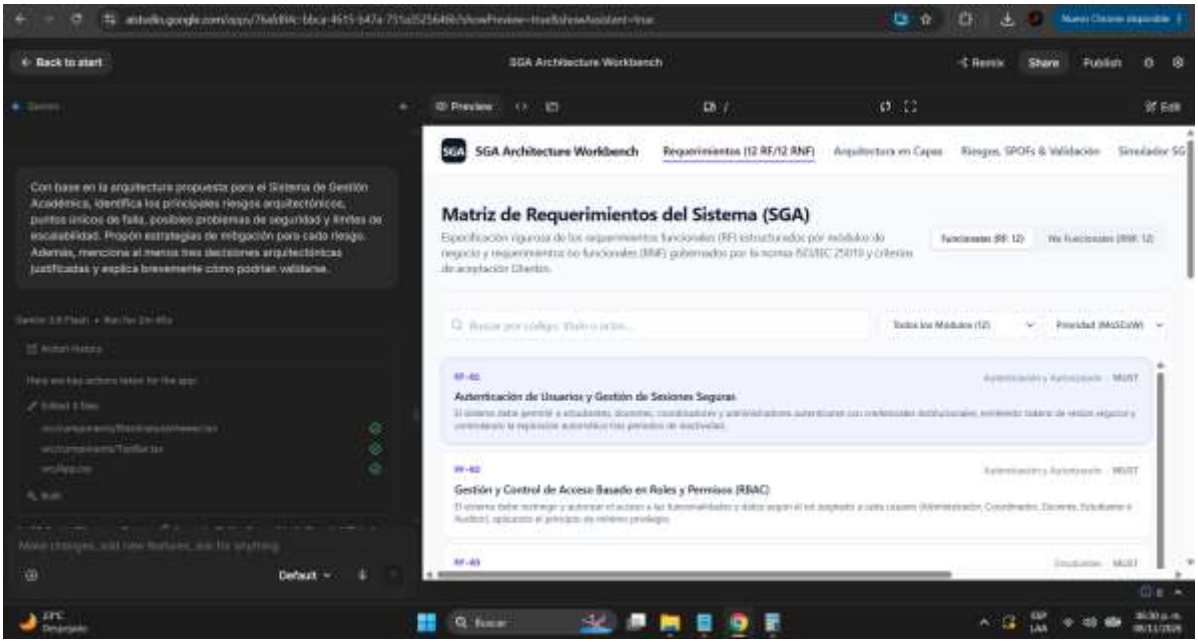
La primera respuesta amplió el alcance definido en la actividad: propuso 17 requerimientos funcionales y 9 no funcionales e incorporó elementos adicionales, como autenticación multifactor y control de acceso basado en atributos. Estos elementos se tomaron como sugerencias de referencia, pero no se adoptaron como requerimientos obligatorios porque no coincidían con los 12 RF y 12 RNF establecidos en la guía del ACA.

Evidencia 2. Corrección de la propuesta y ajuste a los requerimientos oficiales.



En una segunda iteración se indicó expresamente conservar exactamente los 12 requerimientos funcionales y los 12 requerimientos no funcionales del enunciado, sin agregar nuevos. También se orientó la solución hacia una arquitectura en capas con monolito modular, API REST y base de datos relacional. La respuesta corregida muestra 12 RF y 12 RNF, por lo que se consideró coherente con el alcance académico definido.

Evidencia 3. Análisis de riesgos, puntos únicos de falla y validación de decisiones.



La tercera consulta se utilizó para complementar el análisis crítico de la arquitectura. Se solicitaron riesgos, puntos únicos de falla, problemas de seguridad, límites de escalabilidad, estrategias de mitigación y al menos tres decisiones arquitectónicas justificadas. Esta información se contrastó con la propuesta del informe y se empleó como apoyo para las secciones de decisiones arquitectónicas y análisis de riesgos.

11. Enlaces de socialización y repositorio

Video / pitch en YouTube:

Repositorio o espacio de trabajo: https://github.com/GustavoGrajales38/ACA-Arquitectura-Software-SGA

12. Conclusiones y recomendaciones

- La arquitectura en capas con organización modular permite responder al caso sin introducir complejidad operativa innecesaria y facilita la separación entre interfaz, reglas de negocio y persistencia.
- Los modelos elaborados muestran de forma consistente los actores, módulos, infraestructura, datos y flujo de matrícula, permitiendo verificar la cobertura de los principales requerimientos.
- La seguridad, integridad y trazabilidad deben considerarse desde el diseño, especialmente en procesos sensibles como autenticación, matrículas y modificación de calificaciones.
- Los valores de rendimiento, disponibilidad y escalabilidad son objetivos de diseño y deben validarse posteriormente mediante pruebas de carga, monitoreo y ejercicios de recuperación.
- Google AI Studio puede acelerar el análisis y la generación de alternativas, pero sus resultados requieren revisión humana para evitar decisiones sobredimensionadas o inconsistentes con el alcance.

13. Referencias bibliográficas

Bass, L., Clements, P., & Kazman, R. (2021). Software architecture in practice (4th ed.). Addison-Wesley Professional.

Kruchten, P. (1995). The 4+1 view model of architecture. IEEE Software, 12(6), 42-50.

Object Management Group. (2017). OMG Unified Modeling Language (OMG UML), Version 2.5.1.

OWASP Foundation. (2021). OWASP Top 10: The ten most critical web application security risks.

Corporación Unificada Nacional de Educación Superior. (2026). ACA Arquitectura de Software: Diseño de la arquitectura de un Sistema de Gestión Académica mediante Google AI Studio [Guía de actividad].